Contents

小项目与微弹药实施详解（白话版 v4.1） **1**
开工之前：所有小项目共用的纪律 1
名词小词典（按项目分组，v4.1 新增 S5/S7 条目） 1
S1 mini-Raft KV（必做，3-4 周硬顶） 5
S2 C++/io_uring 预取器（并入 P1，不单列） 6
S3 eBPF 观测工具（可选，P1 核心 1 完工之后再做，1-2 周） 7
S4 bitcask mini 存储引擎（v4.1 scope 重写：聚焦 WAL/merge/crash recovery，与 P1 正交） 7
S5 RDMA 传输臂：Soft-RoCE + ibverbs + C++/Go 桥接（2026-10-20 新增，填补空缺序号） 8
S6 K8s 调度插件实战（v4.1 scope 重写：砍阶段 B，聚焦 NFD/拓扑亲和） 10
S7 mini KV Cache Manager（v4.1 新增，P2 互补，2-3 周） 12
微弹药 M1-M5（按天计的练手小品，算不上项目） 13
微弹药与面试域的关系（v4.1 扩展） 15

小项目与微弹药实施详解（白话版 v4.1）

2026-10-15 v4.1 重写：核心变化是**从”二选一”改为”互补而非巩固”**——所有小项目全做但 scope 收窄到大项目没覆盖的空白域；新增 S7（mini KV Cache Manager）、M4（vLLM token 级 profiling）、M5（vLLM 引擎源码精读）专门补 P2 的引擎内核空白；S4 聚焦 WAL/merge/crash recovery（和 P1 的重扫描恢复正交）；S6 砍掉阶段 B（自研最小插件，和 P1 步骤 7 重复）。定位铁律不变：小项目不写进简历，它们的用途只有两个——充当面试弹药，以及给大项目提前扫雷。评价标准只有一条：单位时间内能产出多少面试弹药。

开工之前的纪律（五条）、名词小词典、以及每个项目的完整步骤和验收标准——全部保留原文，只在 scope 调整的地方改。

开工之前：所有小项目共用的纪律

1. 每个项目都要有独立仓库和 README（写清设计取舍、实验数据、复现命令）。代码是写给你自己看的，真正的弹药是 README 和实验记录；
2. 每个项目必须攒齐”弹药三件套”：一个设计取舍、一个亲自踩过的坑、一张数据或对照表；
3. 硬性时间盒：开工之前在 README 顶部写下截止日期；超期了就砍功能，不能砍质量，更不许对自己说”再打磨一周”；
4. 项目做完立刻写一页《复盘卡》：如果重来会怎么改、被追问时最薄弱的三个点。面试之前你只需要看这张卡；
5. 名额纪律（v4.1 改）：**独立小项目可以全做**，不做二选一。但必须遵守互补原则——每个项目只做大项目没覆盖的空白域，不重复。微弹药 M1-M5 另算，总投入不超过 1 周。

名词小词典（按项目分组，v4.1 新增 S5/S7 条目）

S1 相关

名词	大白话解释
Raft	让多台机器对”操作日志的顺序”达成一致的算法，分布式系统的必修课。
任期（term）/ 三种角色	Raft 把时间切成任期；每个节点任一时刻是 leader（主）、follower（从）、candidate（候选人）之一。
leader / follower	主节点接所有写请求；从节点只被动同步。leader 挂了大家重新投票选新的。
脑裂	极端情况下出现两个 leader，各写各的——一致性算法要防的头号事故。
WAL（预写日志）	先把操作追加写到磁盘日志、再应用到内存，崩溃后靠重放日志恢复。
crc32	一段数据的校验码，读回来对不上就说明数据坏了。

名词	大白话解释
提交（commit）	一条日志被多数派节点复制成功，才算”确定下来了”。
线性一致读	读操作要能看到最新的写入。ReadIndex 和 Lease 是实现的两种办法。

S4 相关（v4.1：砍掉 CAS/refcount 相关，聚焦 WAL/merge）

名词	大白话解释
bitcask	一种存储引擎模型：数据只追加写日志文件，索引全放内存。
tombstone（墓碑）	删除操作的标记。bitcask 里删除也是一次写入，写一个特殊标记盖住旧值。
hint file	merge 时生成的”索引提示文件”，让重启后的扫描快得多。
merge（合并）	把多个旧文件里的存活记录整理进新文件，清掉垃圾数据。
WAL replay 恢复	重启时从 manifest 找到最后一次合并位置，replay 之后的 WAL 记录重建索引——**v4.1 新增重点**：和 P1 的重扫描恢复（重启后扫 blocks/ 目录，~ 分钟级）对比，WAL replay 是 ~ 秒级。
LSM / SSTable / compaction	另一类存储引擎（LevelDB/RocksDB 一系）的关键词，S4 只做对读不做实现。

S5 相关（2026-10-20 新增，填补原目录空缺序号）

名词	大白话解释
RDMA（Remote Direct Memory Access）	远程直接内存访问——让一台机器的网卡直接读写另一台机器的内存，全程不经过对方 CPU 和操作系统。核心三大特性：**内核旁路**（网卡直接在用户态下发指令）、**零拷贝**（数据从源内存直接到目标内存，不经过中间 buffer）、**CPU 卸载**（网卡自带传输引擎，处理可靠性/流控）。
ibverbs / libibverbs	RDMA verbs 的用户态库，提供 OpenFabrics RDMA API。核心对象：PD（保护域）、CQ（完成队列）、QP（队列对）、MR（内存区域，注册过的内存才能被 RDMA 访问）。
Soft-RoCE（rdma_rxe）	内核模块，让普通以太网卡也能跑 RDMA 协议栈。**同网段可跑**（不需要 PFC/ECN），跨子网丢包不重发会直接挂死。**功能可验证、性能不可采信**（常常比 TCP 还慢）。
RoCE vs InfiniBand	InfiniBand 是端到端专用协议栈；RoCE 把 IB 传输层搬到以太网链路层，通过普通以太网卡 + 交换机发出去。RoCE 需要 **无损以太网** 三前提。
PFC + ECN + MTU=4096（无损以太网三前提）	RDMA 假设包都到、丢包不重发。RoCE 用 PFC（缓冲区快满时暂停发送，叫”暂停帧”）防止丢包；ECN（队列有点堵时提前标记）让发送方主动降速；MTU=4096（Jumbo Frame）减少分片。三者配合让以太网变”无损”。
RDMA Write vs Send/Recv	Write 是**单边操作**：你只要知道对方内存地址 +rkey，就能直接写过去，对方甚至不知道。Send/Recv 是**双边操作**：对方要先 post receive buffer，配合完成传输。P2P 拉取式场景天然适合 Write（对方无感知）。
rkey / lkey	RDMA 内存注册后，对端拿到的访问 key（rkey）；本端用 lkey。类似”给对方的内存访问令牌”。

名词	大白话解释
CQ 轮询 vs 事件通知	Completion Queue 取事件的两种方式。轮询（`ibv_poll_cq` 不停扫）：延迟最低但 CPU 100%；事件通知（epoll + `ibv_get_cq_event`）：CPU 0% 但 Soft-RoCE 上经常丢事件。**Soft-RoCE 上轮询是被迫选择**。
cgo 不链进 Go 热进程的三个原因	cgo 切换开销 2–5 s（goroutine→C 线程切换），对 RDMA 完成路径不可接受； CQ 轮询放 C 进程里跑更干净，不占 Go runtime 调度； Go GC 可能移动内存，传给 ibverbs 会炸（必须 C.alloc 分配 + 注册）。
Transport 接口抽象	P1 §8.2 把传输层抽象成 Go 的 interface，默认实现是 gRPC stream。RDMA 实验臂再写一个 rdmaTransport，运行时无感知切换。

S2 相关

名词	大白话解释
io_uring	Linux 最新的高性能异步 I/O 接口，一次提交一批读写、一次收割一批结果。
队列深度（QD）	一次排在队列里的 I/O 数量。
O_DIRECT	绕过系统缓存直写磁盘，省一次拷贝，但对内存地址、长度、偏移有对齐要求。
overlayfs	容器镜像用的分层文件系统，在上面用 O_DIRECT 可能直接报错。
fio	磁盘性能测试的标准工具，用来测”裸盘上限”。

S3 相关

名词	大白话解释
eBPF	往内核里安全地挂小程序的机制，观测性能的神器。
tracepoint / kprobe	内核预留的固定观测点 / 可以挂在几乎任何内核函数上的探针。前者稳，后者灵活。
per-CPU map	每核一份数据的内核哈希表，避免多核抢锁。
CO-RE	“一次编译到处运行”：让 eBPF 程序不重编译就能适配不同内核版本。
readahead（预读）	内核猜你要读后面的数据、提前搬进缓存的机制，它会干扰归因。

S6 相关（v4.1：砍掉自研插件相关，聚焦 NFD/拓扑亲和）

名词	大白话解释
scheduler-plugins	Kubernetes 官方的调度器插件仓库，S6 的基座。
扩展点（PreFilter/Score）	调度流程预留的挂钩位置。PreFilter 先筛掉不合适的节点；Score 给剩余节点打分。
NFD / NRT CRD	Node Feature Discovery 负责发现节点硬件拓扑并上报成 NodeResourceTopology 这个自定义资源。
Topology Manager	kubelet 里管 CPU/设备拓扑对齐的组件，v1.27 起 GA。
DRA	Dynamic Resource Allocation，K8s 新一代设备分配机制，v1.35 起 GA。
device plugin（旧模型）	老的设备上报机制，把设备抽象成整数（如 `nvidia.com/gpu: 1`），表达不了拓扑细节。

名词	大白话解释
schedulerName	Pod 上指定”用哪个调度器调度我”的字段，自研调度器靠它接管流量。

S7 相关（v4.1 新增）

名词	大白话解释
vLLM block_manager	vLLM 引擎里管 KV cache block 分配、释放、淘汰、prefix lookup 的核心模块。
block pool	固定大小的 KV cache block 池，从里面分配/回收 block。
BlockHash	block 的递归哈希：`h_i = H(h_{i-1} ‖ token_i)`，天然形成前缀链，用来做 prefix cache 索引。
PrefixHitDepth	新请求能命中缓存里的最长前缀 block 数 × blockSize——vLLM 网关路由的核心依据。
LRU 驱逐	淘汰最近最少使用的 block，同时跳过有 in-flight 请求引用的 block。
ref_count	每个 block 被多少个 in-flight 请求引用的计数，>0 时驱逐不能选它。
vLLM v1 vs v0 的 block manager	v0 用 `BlockTable` 二维数组，v1 用 `BlockPool + Index`——**版本锁定是第一天就要做的事**，接口完全不一样。

M4/M5 相关（v4.1 新增）

名词	大白话解释
eBPF uprobes	挂在用户态函数上的 eBPF 探针，能精确到每个函数调用的时间。
prefill / decode 时间	推理的两个阶段：prefill 把整段输入算一遍吐第一个字（计算密集）；decode 一个字一个字往外蹦（访存密集）。
引擎内核精读	读 vLLM 的 C++/Python 核心代码（block_manager、scheduler、attention），不是路由层。

微弹药相关

名词	大白话解释
namespace（命名空间）	Linux 的隔离机制：PID、网络、挂载点等各隔离一套。容器的地基。
cgroup	Linux 的资源限额机制：CPU、内存用量都能管。
pivot_root / chroot	切换进程根目录的两种手段，前者更彻底。
veth / bridge	一对虚拟网线 / 一台虚拟交换机，容器组网的最小积木。
netns	网络命名空间，一个独立的网络栈。
PV / PVC	持久卷 / 持久卷申领，K8s 里”存储资源”的供需两端。
bind mount	把一个目录原样挂到另一个路径，CSI 挂卷的最后一哆嗦。
CNI / CSI	容器网络接口 / 容器存储接口，两个标准化的插件协议。

S1 mini-Raft KV（必做，3-4 周硬顶）

互补关系：P1 用 etcd（不自研 Raft）、P2 是单机网关——S1 补的是分布式共识的工程经验，这是大项目都不覆盖的能力域。

做什么：用 Go 实现一个 3 节点的 Raft，对外提供 Get/Put 两个接口。目的：把一致性从”背出来的知识点”变成”能讲清楚的东西”。

先保持清醒：这是 MIT 6.824 里人手一个的项目，实现本身的差异化几乎为零，简历上永远不要把它排在第一位；它的弹药价值全在工程细节、实验设计和工业代码对读里。

范围：做选举、日志复制、快照、崩溃恢复；不做 membership 变更和 Pre-Vote（这两个列为”必须能白板讲清楚”的内容）。

步骤 1：单机 KV 加 WAL

先不碰分布式，把单机版本做扎实：1. 实现一个内存 KV 存储（`map[string]string` 加锁即可）；2. 前面挡一层 WAL：每次 Put 先追加写日志、再改内存。日志记录格式 `len | crc32 | entry`，追加写；3. 有三个字段必须持久化：`currentTerm`、`votedFor`、`log[]`。这里埋着经典死亡陷阱：`votedFor` 不落盘，节点重启后就可能在同一个任期里投出两票，进而脑裂；4. 启动时重放 WAL 恢复状态，用 crc 校验识别损坏的尾部记录。

步骤 2：搭 gRPC 骨架和选举

1. 定义两个 gRPC 接口：`RequestVote` 和 `AppendEntries`（后面复制也用它）；
2. 三个角色一个 tick 驱动所有定时器；选举超时在 `[T, 2T)` 区间随机化，T 大约取 500ms——随机化是防止大家同时发起投票；
3. 实现完整的选举流程：follower 超时变 candidate → 给自己投票 → 向所有人拉票 → 多数票当选；
4. 收到每个 RPC 回复都先比较 term，发现自己的 term 更小就立刻让位变 follower——这是第二大陷阱；
5. 日志里打印每次 term 变化和投票流向，调试全靠它。

步骤 3：复制与提交

1. leader 收到写请求 → 追加到自己的日志 → 并行发 `AppendEntries` 给所有 follower → 多数派确认后提交 → 应用到状态机 → 回复客户端；
2. 提交规则：leader 只能提交当前任期的日志（Raft 论文 §5.4.2 的结论，必须能讲为什么）；
3. 日志不一致时的修复：`prevLogIndex/term` 不匹配就回退 `nextIndex` 重发；如果你实现了冲突项的加速回退（一次跳过一整段冲突日志），这是加分项；
4. 把 raftd 集成进测试 harness：能注入延迟、网络分区、进程重启，后面所有实验都跑在这个 harness 上。

步骤 4：快照和 kill -9 恢复

1. 日志太长时做快照：在 `lastIncludedIndex` 之前截断日志，快照保存状态机当前内容；
2. 落后的 follower 直接用 `InstallSnapshot` 传快照，不再逐条补日志；
3. 恢复顺序：先装快照，再重放快照之后的日志；
4. 测试方式简单粗暴：随时 `kill -9` 任意节点再拉起，验证数据不丢、集群能自愈。

步骤 5：v3 的两组实验 + etcd 对读

1. 实验一（换血后的丢包扫描）：丢包率取 0.1% / 1% / 5% / 10% 四档，画 commit 延迟 P50/P99 随丢包率变化的曲线，并解释拐点——重传与选举超时是怎么耦合的。老实验”30% 丢包单点”已删除，它太极端，只能得到”它瘫了”一个结论；
2. 实验二（新增，可用性数字）：`kill -9` leader 至少 100 次，记录从 kill 发生到新 leader 完成第一个成功写入的时间分布（中位数/P99）。分区实验区分三种：多数派隔离、少数派隔离、不对称分区（leader 单向失联），各给一张行为表；
3. 工业对读（必做）：精读 etcd 的 raft 库——重点看 `node.go` 的 Ready 循环、unstable log、Storage 接口、ReadIndex 的工业实现。产出一篇《我的 toy raft 与 etcd raft 的 5 个差异》（候选差异：批量化的 Ready 通道、消息流水线、leader 转移、检查点时机、无环日志结构）；
4. 线性一致读是 stretch 目标：用 ReadIndex 或走日志实现，并能讲清楚”为什么 ReadIndex 对时钟的依赖比 Lease 小”。

验收标准：故障注入下已提交数据零丢失；三类分区行为表完整；丢包扫描曲线画出来；无主窗口分布至少 100 个样本；etcd 差异笔记写完。

你能讲的点：votedFor 为什么必须持久化；为什么只能提交当前任期日志；ReadIndex 与 Lease 的对比；你的 toy 与 etcd 的具体差异；Pre-Vote 如何防止分区节点拿旧 term 扰乱选举；joint consensus 两阶段成员变更（不实现，但要能白板讲）。

S2 C++/io_uring 预取器（并入 P1，不单列）

互补关系：P1/P2 主力 Go，无 C++ 系统编程——S2 补的是 C++ + io_uring + O_DIRECT + aligned memory 这套系统级硬货。同时是 vLLM C++ 源码精读的前置训练（S7/M5 需要读 vLLM 的 C++ 代码，S2 先让你熟悉 C++ 系统编程的坑）。

定位：P1 v3 已经把它收编为 A 轨（小文件高 IOPS 臂）的实验分支。可以单独建仓库，但它不占小项目名额、不另排工期，随 P1 步骤 8 前后推进。

范围：做本地写路径和提交模型；不做完整的 S3 客户端（HTTP Range 已够用）。

步骤 0：锁环境、练手感（v3 新增）

1. `uname -r` 锁定内核版本——registered files/buffers 的行为随版本变化；
2. 把 liburing 官方 examples 逐个通关。io_uring、O_DIRECT、overlayfs 三个坑叠加，直接上手太陡，而 C++ 是你的次语言，这个坡必须先削平。

步骤 1：Go 线程池加 buffered 基线

用 Go 写一个带线程池的 buffered I/O 写入器，跑出第一组基线数字。它是后面所有版本的比较原点。

步骤 2：C++ buffered 版本

同样的逻辑用 C++ 再写一遍。这一步的目的是隔离语言因素：之后 Go 版和 C++ 版的差异里，才能分清哪些来自 io_uring、哪些来自语言本身。

步骤 3：O_DIRECT 加 io_uring

1. 写路径换成 O_DIRECT 直写 + io_uring 异步提交；
2. O_DIRECT 三重对齐的坑：缓冲区地址、长度、偏移都必须对齐逻辑块，用 `posix_memalign` 对齐到 4K；
3. overlayfs 的坑：容器里的 overlayfs 上用 O_DIRECT 可能直接报 EINVAL——目标路径必须落在宿主机真实的 ext4/xfs 上（用 hostPath，或者干脆在宿主机上跑）；
4. 提交与收割：`get_sqe → prep → submit_and_wait(N)`，CQE 批量收割后再 resubmit，错误码逐个处理（短读即 EOF）。

步骤 4：registered 优化版

用 `IORING_REGISTER_FILES` 和 registered buffer 消掉每个操作的固定开销；队列深度在 64–256 之间可配。

步骤 5：跑参数矩阵

队列深度 × 块大小（4K/64K/1M）× 并发扫点，记录吞吐、strace 统计的 syscall 数、CPU 占用。用 fio 先测裸盘直写上限，贴着上限比——“我比 fio 差 8%”这种说法比”我很快”专业十倍。

收益预期预先诚实化：P1 主臂的 4MiB 块经 HTTP 抓取时受网络限速，省 syscall 对吞吐的贡献预期接近零；io_uring 真正的战场是小文件高 IOPS。报告的正确写法：“syscall 数量大幅下降、大块吞吐不变（并解释为什么受网络限速）、小文件臂 IOPS 提升 X”，不许只挑好看的数字报。

弹药：O_DIRECT 对齐与 overlayfs 的坑；io_uring 到底减少了什么（syscall 加线程调度）；为什么大块场景下它赚不到收益——把”不适用的边界”讲清楚，比吹收益值钱得多。

S3 eBPF 观测工具（可选，P1 核心 1 完工之后再做，1-2 周）

互补关系：P1 用用户态 inotify 观测、P2 用 vLLM /metrics 观测——S3 补的是内核级观测，能看到用户态看不到的东西（比如 readahead 造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错）。

定位：原来排在 P1 指标期会跟核心 1 争时间，现在后移；MVP 砍到只剩一个探针。范围只做只读观测；内核版本不低于 5.10；在宿主机上跑（容器权限很麻烦）。

步骤 1：bpf2go 的 hello-world

用 bpf2go 编译并加载一个最小 eBPF 程序，验证环境（clang、内核头、vmlinux.h、CO-RE）全部就绪。跑不通环境，后面免谈。

步骤 2：block I/O 延迟直方图（唯一必做项）

1. 挂两个 tracepoint：`block/block_rq_issue`（下发）和 `block/block_rq_complete`（完成）；
2. 按 request 指针把两次事件配对，算出每个 I/O 的延迟；
3. 在内核侧按 log2 bucket 聚合（别把每个样本都送到用户态）；用户态周期性拉取 per-CPU map；
4. 用 Go 把直方图画出来或导出成指标。

步骤 3（可选）：再挂两个探针

major fault 探针（`exceptions/page_fault_user` 里的 major/minor 位）和 `sys_enter_read` 字节量探针。有余力再做。

步骤 4：接入 P1 实验，交叉验证

1. 把 exporter 接入 P1 的实验流程，与 dataloader 的打点交叉验证口径；
2. 有一条观测逻辑必须写进报告：P1 的 preheat 模型下，缓存命中 page cache 时根本不会产生 block I/O——所以 block I/O 直方图天然就能区分命中与未命中；
3. **readahead 陷阱单独成节**：内核预读造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错。这个陷阱本身就是上等弹药；
4. 准备降级方案：eBPF 挂不上时用采样 `/proc/diskstats` 兜底，并诚实标注两个口径的差异。

验收标准：P1 实验期间能稳定出数；两个口径交叉验证一致；readahead 归因陷阱有专门一节。

S4 bitcask mini 存储引擎（v4.1 scope 重写：聚焦 WAL/merge/crash recovery，与 P1 正交）

互补关系（v4.1 核心）：P1 用 CAS + **重扫描** 做 crash recovery（重启后扫 `blocks/` 目录按路径重建索引，~ 分钟级），不做 WAL、不做 merge、不做增量恢复。S4 补的是 **WAL 持久化 + merge/compaction + WAL replay 增量恢复**（重启后 replay WAL 重建索引，~ 秒级）。面试时的对比故事："P1 的 CAS 存储层靠重扫描做 crash recovery（实现简单，但重启慢），S4 我用 WAL 做增量 recovery（重启快，但 merge 逻辑复杂）——两种 crash recovery 路径我都实践过，知道各自的适用边界。"

做什么：用 Go 实现一个 bitcask：WAL 加内存索引，加 hint file，加 merge。范围：只做单机；不做分布式（S1 已覆盖）、完整的 LSM（只做对读）。

> **v4.1 不再实现**：CAS contentHash 存储层（路径寻址 + 内容哈希摘要）、refcount 引用计数、P2P 块共享——这些是 P1 的东西，S4 不用。S4 用 WAL + CRC 做完整性校验，不做 CAS。

步骤 1：append-only log 加读路径

1. 记录格式：`crc32 | tstamp(8B) | keysz(4) | valsz(4) | key | value`。时间戳统一用 8 字节 int64（毫秒/微秒）——4 字节秒级时间戳有 2038 问题，主动在注释里写出这个修正，小细节但能留下大印象；
2. 删除操作也是一次写：tombstone 用 valsz 的特殊值标记；

3. 内存索引：`map[key] → {fileID, offset, size}`；
4. 读路径：索引命中后用 `pread` 读取，再做 crc 校验——校验失败意味着盘上数据腐坏，这时报错或触发重新 merge。

步骤 2：多文件轮转加 tombstone

1. 活跃文件写满 256MB 后转为 immutable，新开一个活跃文件接着写；
2. fsync 三档做成配置：逐条刷盘 / group commit（N 毫秒）/ 只靠操作系统。三档拿来 benchmark 对比，这是存储引擎最核心的权衡之一；
3. 明确实现单写多读模型：写锁天然串行化（这是 bitcask 和 LSM 共同的核心约束），读操作无锁；后面压测要验证多写者的排队情况和多读的扩展性。

步骤 3：启动恢复对比（v4.1 新增，和 P1 正交核心）

1. **暴力扫全目录**（基线）：启动时把所有数据文件从头到尾读一遍重建索引；
2. **WAL replay**（v4.1 重点）：启动时从 manifest 找到最后一次合并位置，replay 之后的 WAL 记录重建内存索引；
3. **启动扫描 + hint file**：merge 时顺手生成的索引提示文件，启动只读它；
4. 三种方式的启动时间做成对比数据——预期是 replay 秒级、重扫描分钟级、hint file 介于两者之间。这张对比表就是和 P1 正交的核心弹药。

步骤 4：merge 加 manifest 原子切换

1. merge：把 immutable 文件中的存活记录写入新数据文件和 hint file；
2. 原子切换的顺序：先 rename 数据文件和 hint 文件，最后写 manifest 指向新的文件集合；
3. 把每一个崩溃窗口想清楚、写进注释：rename 到一半崩溃会怎样？manifest 写一半呢？旧文件什么时候才能删？

步骤 5：fuzz 崩溃矩阵

用 fuzz 随机注入 `kill -9`（在写、在 merge、在 rename 等各种时机杀），跑完崩溃矩阵，验证任何时刻重启都能恢复到一致状态。

步骤 6：benchmark

1. 1M/10M 两档都要跑（32G 内存的机器跑得动 10M 档）：1M 档算不出 bitcask 的命门——全内存索引的容量账。实测启动时间、常驻内存、读写吞吐，画出”索引内存/数据量”的比值曲线；
2. merge 期间的写延迟抖动——再丑的数字也要敢贴；
3. 写放大计算（写入磁盘的字节数 ÷ 有效数据量）；
4. fsync 三档对比曲线；
5. 多写者排队与多读扩展性压测；
6. **v4.1 新增：启动恢复时间对比表**：把 P1 的重扫描恢复数据和 S4 的 WAL replay 数据放在同一张表里，量化两种 crash recovery 的差异（这个可以等 P1 的重扫描做完之后补）。

步骤 7：LSM 对读（不实现）

读 LevelDB/RocksDB 的 compaction 文档，最终目标是在白板上回答：“数据量超过内存之后，bitcask 为什么会输给 LSM?”（对比 SSTable、稀疏索引、分层 compaction 的读写放大）。

验收标准：崩溃矩阵全绿；10M 档容量账有实测；merge 抖动数据完整；**启动恢复时间对比表**有实测；LSM 对读笔记一页。

弹药三件套：- 取舍：WAL replay vs 重扫描（实现复杂度 vs 恢复速度）；merge 原子切换的崩溃窗口；fsync 三档的性能 vs 持久化保证 - 坑：overlayfs 上 O_DIRECT 的坑（和 S2 联动）；merge 期间的写延迟抖动 - 数据：10M key 内存账；fsync 三档吞吐对比；**启动恢复时间对比表**（P1 重扫描 vs S4 WAL replay）

S5 RDMA 传输臂：Soft-RoCE + ibverbs + C++/Go 桥接（2026-10-20 新增，填补空缺序号）

> 源自 P1 §9.1 RDMA 实验臂——P1 v4 正文里只写了”永远不进主路径”就没了，Transport 接口抽象出来了但 RDMA 那个实现没人写过。S5 把这块独立成完整实施指南。**P1 PDF 保持不变**。

互补关系：P1 的主路径用 gRPC stream 做数据传输（§8.2 Transport 接口默认实现），简单可靠、生态成熟、调试方便。RDMA 作为实验臂挂在 P2P 块共享后面，v4 正文里只写了"永远不进主路径"就没了——Transport 接口抽象出来了，但 RDMA 那个实现没人写过。S5 补的是：真正手写 ibverbs 代码跑通 RDMA 数据面、通信原语选型决策（Send/Recv vs RDMA Write）、以及 Go C++ 跨语言桥接。这三件事 P1 里只提了一句"别用 cgo 链进 Go 热进程，用独立 C++ 组件加 C ABI"，但为什么这样、桥接怎么设计、Go 侧指针怎么处理——全是空白。

面试时的对比故事："P1 用 gRPC stream 做主力传输（可靠、好 debug、生态全），S5 我用 RDMA verbs 写了 P2P 实验臂（内核旁路、零拷贝、CPU 零开销，但工程复杂度完全不是一个量级）——两套传输栈我都写过，知道什么时候该用哪个，也知道 RoCE 在跨子网时的无损以太网坑。"

做什么：在两台 Linux 虚拟机上用 Soft-RoCE（内核模块，不需要买 Mellanox 网卡）跑通真正的 RDMA verbs 代码，实现一个对标 P1 Transport 接口的块数据传输原型。把 P1 里"挂个 Transport 接口"从设计图变成真正跑在网卡上的数据通路。

范围：只做 Soft-RoCE 功能验证；不做 GPUDirect RDMA（没有 GPU）；不进 P1 主路径（RDMA 传输臂永远是 P2P 的可选后端）。

不做：eRDMA 云真机校准（真值数据，按小时租阿里云 eRDMA 卡一下就行，几十块钱，不属于本项目工期）、GPUDirect RDMA、RDMA 进 P1 主路径替换 gRPC。

步骤 1：Soft-RoCE 环境搭建（2 小时，闸门）

两台 Ubuntu VM（内核 5.x），加载 softroce 模块，配好同网段 IP，用 `rping -s 12345 -c` 跑通 RDMA Ping。这一步是后面所有事的闸门——rping 不通就别往下走。

1. 两台虚拟机（或容器，Docker 里需要 `--cap-add=NET_ADMIN`）；
2. `sudo modprobe softroce`（内核 5.x 自带）；
3. 配置同网段 IP（如 192.168.1.1 和 192.168.1.2）；
4. `rping` 双向跑通，确认 RDMA 数据面真的在工作。

步骤 2：最小 ibverbs demo（半天）

纯 C++ 写一个 client/server，跑通 RDMA Write。不需要 Go，先把 ibverbs API 本身搞定。重点理解四个核心函数：

1. `ibv_create_qp` / `ibv_qp_rig_init`：创建队列对并从 INIT 状态经过 RTR→RTS 到就绪；
2. `ibv_post_send`：下发 RDMA Write 操作；
3. `ibv_poll_cq`：轮询 completion queue 等待完成；
4. `ibv_reg_mr`：注册内存区域（register memory region），让网卡能直接访问。

client 注册内存、post send（`opcode=IBV_WR_RDMA_WRITE_WITH_IMM`），server 注册内存、等待 completion。成功的标志：server 那边的内存确实被改了。

步骤 3：C ABI 封装 + Go 桥接（1-2 天）

把 ibverbs 代码封成一个共享库，暴露 3-4 个简单函数，Go 侧用 `import "C"` 调用。**核心约束：不要把 ibverbs 的任何内部状态暴露给 Go 侧**——Go 只管调高层接口，所有复杂状态（PD/CQ/QP/MR）都在 C++ 里。

暴露的 C ABI：

```
// 打开设备、创建 PD/CQ/QP、预注册 MR（几块几 MB 的大块槽位）
int rdma_init(const char* device_name, int port);
// 发起 RDMA Write 到对端（rkey + addr 来自对端）
int rdma_write(uint32_t peer_rkey, uint64_t peer_addr, const void* local_buf, size_t len);
// 阻塞等待完成，timeout_ms，返回 0 成功 / 超时返回 -1
int rdma_poll(int timeout_ms);
// 关闭设备、清理资源
void rdma_shutdown();
```

Go 侧调用这些函数，但 **所有传给 `rdma_write` 的 buffer 必须来自 C 分配的内存**（`C.malloc(len)` + `ibv_reg_mr` 在 C 侧注册好），不能用 Go 的 `make([]byte)`——Go GC 可能移动内存，RDMA 操作会访问到脏地址。

步骤 4：对标 P1 Transport 接口（半天）

把 C ABI 包装成 Go 的 Transport 接口实现。P1 §8.2 里 Transport 是 Go 的 interface，默认实现是 gRPC stream。你要再写一个 `rdmaTransport` struct，实现同样的 `SendChunk` / `ReceiveChunk` 方法，内部调 C ABI。P2P 块共享在初始化时注入 Transport 实现——gRPC 或 RDMA，运行时无感知。

步骤 5：P2P 块传输原型（1 天）

两台节点各起一个 RDMA 守护进程（C++）+ 一个 Go peer 服务：

1. 节点 A 预注册 8 个 1MB 块槽位的 MR，把 rkey+addr 发给节点 B（走 gRPC 通道交换，这是唯一需要 gRPC 的地方）；
2. 节点 B 收到"需要块 X"的请求，用一条 RDMA Write 把本地块写到 A 的槽位里；
3. 写完后 B 的 completion queue 通知守护进程完成；
4. A 的守护进程从 CQ 轮询到事件，通知 Go 侧"块 X 就绪"。

这就是 P1 P2P 块共享场景下 RDMA 实验臂的完整数据通路。

步骤 6：性能对比实验（半天）

Soft-RoCE vs TCP（Go net 标准库）vs gRPC stream，三档并排对比：块大小 4KB / 256KB / 4MB，每档跑 1000 次取 P50/P99 延迟和吞吐。

诚实预期：Soft-RoCE 常常比 TCP 还慢（因为是软件模拟），但你要把数据贴出来——这恰好是面试时"我知道 RDMA 在什么场景下才真正有价值"的弹药。

另外测两组：1. CQ 轮询 vs 事件通知的 CPU 占用差异；2. `ibv_reg_mr` 注册耗时（注册 4KB 小块 vs 注册 4MB 大块，差多少——证明为什么不能按需注册）。

步骤 7（可选，不在工期内）：租阿里云 eRDMA 校准真值

Soft-RoCE 是模拟器，性能数字不能进报告。真值数据按小时租阿里云 eRDMA 云主机，几十块钱就能拿到。校准一次就够了——对比 Soft-RoCE 和真 eRDMA 的延迟差距（可能几十倍），这个差距本身就是弹药。但这一步不占工期，放在最后按预算推进。

验收标准：Soft-RoCE 下 RDMA Write 功能跑通（Step 5 的 P2P 块传输原型）；4KB / 256KB / 4MB 三档延迟吞吐数据完整；Go C++ 桥接代码有注释说明为什么不用 cgo 链进 Go 热进程；RoCE 无损以太网的 PFC / ECN 机制能讲清楚；`ibv_reg_mr` 注册耗时有实测数据（证明为什么必须预注册大块内存）。

弹药三件套：- 取舍：Send/Recv vs RDMA Write（双边 vs 单边，对端是否需要配合，P2P 拉取式场景天然适合 Write）；CQ 轮询 vs 事件通知（延迟最低 vs CPU 零占用，但 Soft-RoCE 上事件通知经常丢，轮询是被迫选择）；cgo 链进 Go 热进程 vs 独立 C++ 组件（工程复杂度 vs 性能与稳定性，P1 明确指定后者）。- 坑：Go 分配的内存传给 `ibv_reg_mr` 会被 GC 移动，导致注册无效（解决方案：用 `C.malloc` 分配，或 Go 1.21+ 的 `runtime.Pinning` 固定内存）；Soft-RoCE 跨子网丢包不重发（不支持 PFC/ECN 时 RDMA 直接挂死）；`ibv_reg_mr` 按需注册 4KB 小块的开销比 TCP 还大（必须大块预注册 + 内存池）；CQ 事件通知在 Soft-RoCE 上丢事件导致请求挂起超时（只能轮询）。- 数据：Soft-RoCE vs TCP vs gRPC 三档延迟吞吐对比表；`ibv_reg_mr` 注册耗时 vs 块大小曲线（证明预注册大块的必要性）；CQ 轮询 vs 事件通知的 CPU 占用对比；（可选）Soft-RoCE vs eRDMA 云真机的延迟差距。

S6 K8s 调度插件实战（v4.1 scope 重写：砍阶段 B，聚焦 NFD/拓扑亲和）

互补关系（v4.1 核心）：P1 步骤 7 只做 **数据亲和**（前缀就绪度门控 + 打分 + SchedulingGate），没有 NFD/topology-updater/拓扑约束。S6 补的是 **拓扑亲和**——从硬件拓扑（NUMA/GPU zone）角度做 Filter/Score，和 P1 的数据亲和是正交的两个调度维度。面试时的对比故事："P1 做了数据亲和调度（前缀就绪度门控 + 打分），S6 我补了拓扑亲和调度——用 NFD 抓 NUMA 拓扑，用官方 NRT 插件做 Filter/Score，然后在 P1 的数据亲和插件上加了一列拓扑维度，做了双目标冲突实验。两条正交的调度维度我都有实践。"

做什么：基于 kubernetes-sigs/scheduler-plugins，跑通官方的拓扑感知调度，在"数据亲和 × 拓扑亲和"的冲突点上做增量实验。绝对不要声称"我发明了拓扑调度器"，官方 NRT 插件就是你的对照组。

范围（v4.1 修改）：- **阶段 A**（保留，独立工期 ~1 周）：部署 NFD + topology-updater + 官方 NRT 插件全链路 - **阶段 B**（砍掉）：自研最小 PreFilter + Score 插件——和 P1 步骤 7 的数据亲和插件完全重复（P1 步骤 7 已经写了更完整的版本：数据亲和 + SchedulingGate + 门控 + 降级），再写一遍是重复劳动 - **阶段 C**（保留，与 P1 步骤 7 合流，不额外占工期）：双目标冲突实验

开工第一件事：硬件诚实清单

1. 先跑一次 `numactl -H` 确认：游戏本的消费级 CPU 几乎肯定是单 NUMA 节点，也没有多 GPU；
2. 所以所有拓扑都用 fake/annotation 驱动来开发——这完全正当，官方插件自己的 e2e 测试也在 mock NRT 对象；
3. 真机多 GPU 拓扑只在 P2 租云主机的那个窗口顺带读一次（`nvidia-smi topo -m`），不值得单独开预算。

DRA 白板要点（不写代码，与阶段 A 同期写半页笔记，四条必须脱稿讲）

1. 旧 device plugin 模型的痛点：设备被抽象成整数资源（如 `nvidia.com/gpu: 1`），没法按显存、型号或互联拓扑发结构化请求，不支持设备共享，拓扑信息只能走 Topology Manager + NFD + NRT 这条外挂链路；
2. DRA 的四个核心对象：驱动用 ResourceSlice 上报设备能力与拓扑；管理员定义 DeviceClass（类比 StorageClass）；工作负载用 ResourceClaim / ResourceClaimTemplate 申领设备（体验类似 PVC/PV）；调度器做结构化匹配，kubelet 与驱动完成分配握手。支持 CEL 属性过滤和设备共享；
3. 它与 NRT 的关系：DRA 让设备拓扑成为调度器可直接消费的结构化属性，长期会收编 NFD+NRT 链路的一部分；但迁移期两套并存，NRT 插件在现网仍被广泛部署。能讲清”哪部分会被取代、哪部分（NUMA/CPU 亲和）仍会独立存在、迁移卡在哪（驱动成熟度与存量集群版本）“才是满分答案；
4. 现状判断(2026-10 核实)：DRA 从 v1.35 起 GA(stable)，v1.36 把 ResourceClaim 接入了 Workload/PodGroup 体系。禁止再说”DRA 是未来的实验特性”；v1.36 的 gang scheduling 与 P1 的训练 Job 调度是同一个演进方向，当谈资，不跟进实现。

可选的两小时手感练习（不过夜、不写简历、不挤占 P1）

阶段 A 完成之后，用 kind 建一个 1.35 以上的集群，跑官方的 `dra-example-driver`（不需要真实硬件的假设备驱动），创建一个 DeviceClass 加一个 ResourceClaim，`kubectl describe resourceclaim` 能看到分配结果和 ResourceSlice 内容就收工。目标只是”见过真的 ResourceSlice 长什么样”。

阶段 A：现勘并跑通官方链路（约 1 周，v4.1 保留）

1. 搭好 kind 集群（版本对齐 scheduler-plugins 的兼容矩阵），按仓库 Developer Guide 在本地构建镜像（`kind load`）或直接用发布的 chart；
2. 部署 NFD 加 topology-updater，生成真实的 `NodeResourceTopology` CR；在单 NUMA 机器上手写一份 fake NRT yaml（构造双 NUMA、fake GPU 资源池），作为第二个数据源；
3. 配置 v1 版本的 `KubeSchedulerConfiguration`：profiles (schedulerName)、plugins（在 filter/score 上启用 NRT 插件）、pluginConfig (scoringStrategy: MostAllocated、BalancedAllocation、LeastAllocated 各跑一遍，观察决策差异）；
4. 部署若干请求 CPU/GPU 的测试 Pod，验证绑定与拓扑决策符合预期，抓取调度日志；
5. 精读官方 Score 插件源码（分数如何由 NRT 的 zones 和资源分配算出、如何归一化）和 NRT CRD 的字段语义；
6. **验收**(能口述全链路)：device plugin 的 TopologyInfo → kubelet Topology Manager → NFD topology-updater → NRT CRD → 调度插件 Filter/Score → bind；
7. 风险保护：一周内官方全家桶部署不成功（NFD 组件多、镜像坑多），S6 立即降级为”手写 NRT yaml + 只部署插件本体”；再不行降级为”纯跟读源码 + 写决策日志推演笔记”，不能让环境黑洞吃掉第二周。

阶段 B：自研最小插件（v4.1 砍掉，理由）

> 为什么砍掉：P1 步骤 7 要写的调度插件比 S6 阶段 B 的”最小件”还完整——P1 有数据亲和 + SchedulingGate + 门控 + 降级，S6 阶段 B 只有拓扑亲和的最小件。做了 P1 步骤 7 之后，再做 S6 阶段 B 等于再写一遍同一个 framework 的同一种插件，只是打分公式换成另一种。面试时只能讲”我写过两个调度插件”，讲不出差异。

阶段 C：双目标冲突实验（1-2 周，与 P1 步骤 7 合流，不额外占工期，v4.1 保留）

1. 让两个 Score 维度并存：**数据亲和分**（P1 的 Job 子集 readyRatio，由 P1 的 informer 缓存提供）× **拓扑亲和分**（fake NUMA/GPU zone 匹配度，由 S6 阶段 A 的 NRT CRD 提供）；
2. 构造冲突场景：Job 的热分片缓存在 NUMA-A 节点上，但空闲 GPU 在拓扑上更远的节点；或者另一个节点 NUMA 局部性很好但数据是冷的；
3. 权重 w 从 0 扫到 1，画出权重-调度决策曲线，以及对应的 makespan/拓扑惩罚代理指标；
4. 把终点问题写成报告：框架级多插件的分数如何归一化（各插件 Score 的值域不同）、冲突时为什么不存在银弹、生产环境应该让谁把权重暴露给用户；
5. 合流动作：**S6 的拓扑亲和 Score 直接作为 P1 步骤 7 调度插件的第二个 Score 维度**——P1 仓库里保留完整的双维度插件，S6 仓库里保留阶段 A 的部署脚本、阶段 C 的实验报告和权重扫描曲线；
6. **验收**：产出一张冲突权重-决策曲线图；DRA 四条白板要点能脱稿讲。

弹药三件套：- 取舍（v4.1 修改）：NRT annotation fake 与 NFD 全家桶之间怎么选、双目标如何归一化、**为什么阶段 B 被 P1 步骤 7 覆盖了**（体现你能识别重复劳动、主动优化）- 坑：KubeSchedulerConfiguration 的 apiVersion、多 profile、schedulerName、RBAC 权限、kind 镜像加载 - 数据：权重-决策曲线加 scoringStrategy 对照

诚实清单：拓扑全部是 fake 的；官方 NRT 插件是基线，并不是要打倒的对手；没有做绑核验证，没有真机性能数据；DRA 没有动手实现，但它的现状（v1.35 已 GA）和四个要点必须跟到能讲、能答迁移期共存问题的程度。

S7 mini KV Cache Manager（v4.1 新增，P2 互补，2-3 周）

互补关系：P2 把 vLLM 当黑盒用（只消费 KV 事件，不自研 KV block 管理）。S7 补的是 **引擎内核视角**——精读 vLLM 的 `block_manager` 之后，用 C++ 写一个简化版 block manager，管理固定大小的 KV cache blocks、LRU 驱逐、prefix lookup、capacity 动态调整。面试时的对比故事："P2 是网关视角看 KV cache（消费引擎推的事件），S7 是引擎内核视角看 KV cache（自己管理 block pool、驱逐、prefix 匹配）——两种视角我都有实践，知道引擎内部和网关之间怎么协作。"

时间盒：2-3 周（和 P2 步骤 4 并行）

前置：S2 的 C++ 系统编程经验（io_uring / O_DIRECT 之后你对 C++ 内存管理更熟了）、M5 的 vLLM 源码精读笔记（先读再写）

vLLM 内核精读（前置，M5 联动）

精读 vLLM 源码的三个核心文件（用 vLLM v1.x 锁定版本）：1. `vllm/v1/core/kv_cache_manager.py`（或 `vllm/vllm/v1/worker/gpu_model_runner.py` 里的 block manager）：KV block 的分配、释放、淘汰、prefix cache 索引 2. `vllm/attention/backends/paged.py`：paged attention 的内存布局（每个 block 多大、token 怎么映射到 block）3. `vllm/vllm/v1/core/scheduler.py`：continuous batching 调度时 KV cache 怎么被引用和释放

产出一篇《vLLM KV Cache 内核对读笔记》——重点记录三个问题：- block 是多大？token 怎么映射到 block index？- LRU 驱逐时怎么判断"这个 block 还能不能被某个 in-flight 请求引用"？- prefix cache 的索引结构是什么？怎么判断"新请求的前缀命中了缓存里的哪些 block"？

C++ 简化版实现

精读完 vLLM 内核后，用 C++（C++17，不用 CUDA，跑在 CPU 上）写简化版 block manager：

核心数据结构：- `Block`：固定大小（默认 16 blocks/页，可配置），每个 block 存一段 token 的 KV 数据（简化为固定大小的 buffer，不需要真 FP16/BF16 数据）- `BlockPool`：`vector<Block>`，固定容量，初始化时全部标记为 free - `PrefixIndex`：`map<BlockHash, vector<BlockID>>`——block 哈希到持有该 block 的 ID 列表 - `LRU`：`list<BlockID>` + `unordered_map<BlockID, list::iterator>`

核心接口：

```
class KVCacheManager {
public:
    vector<BlockID> AllocBlocks(int n);              // 从 pool 分配 n 个连续 block
    void RefBlocks(const vector<BlockID>&, RequestID); // 标记 in-flight 引用
```

```
    void UnrefBlocks(const vector<BlockID>&, RequestID); // 释放引用
    int EvictLRU(int n);                                  // LRU 驱逐，跳过有引用的 block
    int PrefixHitDepth(const vector<BlockHash>&);        // 最长前缀命中深度
    void ResizePool(int new_capacity);                    // 动态调整容量
};
```

关键设计：- BlockHash 对齐 vLLM：按前 ~256 token 一个 block 的粒度做递归哈希 `h_i = H(h_{i-1} ‖ token_i)` - in-flight ref_count：每个 block 有一个 `ref_count`，> 0 时 LRU 跳过；这是 vLLM 驱逐时不驱逐活跃请求 block 的核心机制 - PrefixHitDepth 语义：从第一个 block hash 开始连续匹配，断开后不计入（和 P1 的 prefixLen 语义一致）- 线程安全：用 `shared_mutex`——读路径（PrefixHitDepth）用读锁，写路径（Alloc/Ref/Unref/Evict）用写锁

实验与对读

实验（和 P2 的负载台联动）：1. LRU 驱逐曲线：固定 pool 容量，用 P2 的负载台 trace 跑 1000 个请求，记录驱逐命中率（被驱逐的 block 之后是否被重新引用）、in-flight block 占比随时间变化 2. Prefix lookup 精度：构造三种场景（全命中、半命中、全 miss），对比简化版和 vLLM 真实实现的 prefix hit depth 数值——简化版的 hash 算法和 vLLM 不一样，所以数值会有差异，但**曲线形状应该一致** 3. Capacity 动态调整：模拟弹性伸缩——pool 从 512 → 1024 → 512 → 2048，观察驱逐和 prefix hit depth 的响应

工业对读（必做）：精读 vLLM 内核里的驱逐逻辑——重点看它怎么处理”这个 block 还在 in-flight 请求里但又没被显式 ref”的边界情况。你的简化版用显式 Ref/Unref 处理，vLLM 内部是怎么处理的？差异有多大？

验收标准：Block pool 全功能可用（Alloc/Ref/Unref/Evict/PrefixHitDepth/Resize）；LRU 驱逐曲线 + prefix lookup 精度对比 + capacity 调整曲线三张图；《vLLM KV Cache 内核对读笔记》写完，能讲清楚简化版和 vLLM 内核的 3 个核心差异。

弹药三件套：- 取舍：为什么 vLLM 用显式 ref_count 管理 in-flight block（而不是靠 GC 或所有权）；PrefixHitDepth 的语义为什么是连续匹配而不是覆盖率 - 坑：vLLM v1.x 的 block manager 接口和 v0.x 完全不一样（v0.x 用 `BlockTable` 二维数组，v1.x 用 `BlockPool + Index`），**版本锁定是第一天就要做的事** - 数据：LRU 驱逐命中率曲线；**简化版 vs vLLM 内核差异对照表**

微弹药 M1-M5（按天计的练手小品，算不上项目）

定位纪律：1. 它们不算项目：一个目录加一份 NOTES.md 就够，不做 README 包装、不写进简历、不占小项目名额。“我写过、代码就在这里、原理我讲得清楚”就是真弹药；2. 硬性时间盒：单个不超过 2-3 天，总投入不超过 1 周；一旦超时，立即走该条目的降级路径，不过夜、不打磨；3. 窗口纪律：只允许在寒假、实习空档或两个大项目的里程碑之间做；P1 核心 1 冲刺期（步骤 7 前后）禁止开工；4. 顺序：M1 所有人必做；M2/M3/M4/M5 按岗位选——面存储岗优先 M3；面平台/调度岗优先 M2；面推理引擎岗优先 M4/M5。

M1 mini-container（必做，1-2 天，性价比最高）

做什么：用 Go（或直接用 C 调 `clone`）写一个 100-200 行的容器运行器。- 用 `clone` 加 `CLONE_NEWPID / NEWUTS / NEWNS` 三个标志创建子进程（有余力再加 `NEWNET/NEWUSER`）——它就有了独立的进程号空间、主机名和挂载表；- cgroup 用 v2：在 `cgroup.controllers` 体系下写 `cpu.max`、`memory.max`、`memory.swap.max`，把子进程的 pid 写入 `cgroup.procs`，资源限额就生效了；- rootfs 用 busybox 或 distroless 的根目录，调用 `pivot_root` 切换（入门版可以先用 `chroot`），再挂载 `/proc`、`/tmp`；- stretch（可选）：用 overlayfs 的 lower/upper/merged 做一个可写层——正好与 S2 里”overlayfs 上 O_DIRECT 的坑”呼应。

验收标准：容器内执行 `ps` 只能看到自己的进程树；hostname 相互独立；把 `memory.max` 设成 64M 后跑内存分配，能触发 cgroup OOM 并观察宿主行为；用 `lsns` 能看到新的命名空间。

你必须能讲：容器为什么不是虚拟机（共享宿主内核意味着什么、攻击面在哪里）；cgroup v2 相对 v1 的统一层级变化；pivot_root 与 chroot 的区别；为什么 user namespace 是 rootless 容器的关键；containerd/runC 在 K8s 链路中的位置（kubelet → CRI → containerd → runC）。

降级路径：`pivot_root` 卡住就退回 `chroot` 加 mount namespace，核心目标（找到 namespace/cgroup 的手感）不打折。

M2 mini-CNI（平台/网络向，1-2 天）

做什么：写一个能被 CNI 规范真正调用起来的最简网络插件（Go 可执行文件）。- 实现 CNI 规范的 ADD / DEL / CHECK 三个命令：从 stdin 读 NetConf JSON，从 `CNI_ARGS` 和环境变量里拿 netns 路径；- bridge 模式：建好 bridge，给容器 netns 创建 veth pair 并把一端塞进去，从 `/24` 地址池分配 IP（用位图或租约文件持久化，防止 ADD 重启之后地址泄漏），配好接口地址与默认路由，再把 lo 拉起来；- 用 `cnitool` 在两个独立的 netns 上验证互通；DEL 时释放 IP、删掉 veth。

验收标准：两个 netns 之间能 ping 通；重复 ADD 不会撞 IP；DEL 之后地址正确回收；用 `tcpdump` 能抓到 bridge 转发的包。

你必须能讲：K8s 的”IP-per-pod”网络模型；CNI 在容器创建链路里哪个时刻被调用（netns 已建好、rootfs 已就位之后，由运行时/CRI 调用）；跨节点流量怎么办（VXLAN、路由、BGP——这个练习故意不做，但你要能说清边界在哪里）；service 的 iptables/IPVS 与 CNI 为什么是两层东西。

降级路径：CNI 调试吃掉半天还没结果就停下，改为读 `containernetworking/plugins` 的 bridge 插件源码，再用 `ip link/addr/route` 手敲复现全链路，然后写笔记。

v4.1 扩展（2026-10-20 新增，+ 半天）：手写 iptables/IPVS 规则做一个 mini-Service。

为什么要加：原来”Service 的 iptables/IPVS 是另一层”只是嘴上说的边界。面试官追问”你说那是另一层，那你做过那层吗”的时候，这个扩展就是你的货。在 M2 已经跑通的 netns 环境上继续做：

- 定一个 ClusterIP（比如 `10.96.0.10:80`），手写 iptables 的 DNAT 规则，把去这个地址的流量转到后端：注意 `PREROUTING` 和 `OUTPUT` 两条链都要挂——从本机自己发出的包不走路由决策，这是第一个必踩的坑；
- 用 `-m statistic --mode random --probability 0.5` 挂两条 DNAT，分别指向两个后端 netns 的 IP——这正是 kube-proxy iptables 模式的微缩版（真集群里是 KUBE-SERVICES → KUBE-SVC-xxx → KUBE-SEP-xxx 三层链跳，按概率分摊到各个后端）；
- 两个后端 netns 里各跑一个 `python3 -m http.server`，从宿主 curl ClusterIP 100 次，再用 `iptables -t nat -L -v -n` 看每条规则上的 pkts 计数——统计随机不是严格的 1:1；
- 换 IPVS 模式再来一遍：`ipvsadm -A -t 10.96.0.10:80 -s rr` 加两个 real server（`-m` 即 masq 模式），同样 curl 100 次，对比 rr 的严格轮询和 iptables 统计随机的分布差；
- 量级实验：在 DNAT 规则前面塞 500 / 1000 / 5000 条 dummy 规则，用 wrk 或 ab 压两种模式的延迟——iptables 是一条条顺着匹配（O(n)），IPVS 是哈希表直查（O(1)）。单机量级很可能看不出差别，**阴性结果照样写进报告**，这反而是”真测过”的证据。

扩展验收：ClusterIP 能轮询命中两个后端；iptables 计数器和 `ipvsadm -L --stats` 的分布对比有实测数字；dummy 规则实验有一张延迟表（哪怕是平的）。

扩展后必须能讲：kube-proxy iptables 模式的三层链跳结构，以及为什么用 statistic random 而不是严格轮询；DNAT 在 netfilter 哪个钩子上发生、conntrack 在这里记了什么（回包怎么找到回来的路）；iptables 链式遍历 vs IPVS 哈希表在 Service 数量大时的差别；nftables 解决了 iptables 哪些工程痛点（这次不做，能讲清边界就行）。

M3 mini-CSI（存储/数据流向，2-3 天，与 P1/S4 咬合最紧）

做什么：写一个最小的 CSI 驱动，让它真的被 kubelet 调起来，让 Pod 能挂载你提供的卷。- 实现三个 gRPC service：Identity（GetPluginInfo/Capabilities/Probe）、Controller（最小实现，先不做 provisioning）、Node（在 NodePublishVolume 里把宿主目录 bind mount 到 kubelet 给的 target path；NodeUnpublishVolume 反向卸载）；- 走 kubelet 的 plugin registration 机制：在 `/var/lib/kubelet/plugins_registry/` 暴露注册 socket，与 kubelet 完成握手——这一步本身就是最有价值的弹药；- 先打通 static PV 加你的 CSI driver 的绑定挂载闭环；stretch 目标：实现 CreateVolume/DeleteVolume（在宿主上建/删目录）加 StorageClass 动态供给；- 部署形态二选一，先跑通简单的那种：kind 里用 DaemonSet（privileged、挂 hostPath），或者直接作为节点进程跑。

验收标准：PVC 完成绑定 → 测试 Pod 挂载成功并能读写文件 → 删掉 Pod/PVC 之后 bind mount 被干净卸载（用 `findmnt` 验证没有残留）。

你必须能讲：CSI 三段式流程——Attach（ControllerPublish）、Stage（NodeStageVolume）、Publish（bind mount 到 Pod）各自解决什么问题，为什么最简实现只需要 Publish；kubelet-registration 握手与 socket 的生命周期；ephemeral 卷与 persistent 卷的区别；Fluid/JuiceFS 为什么需要自己的 CSI driver（FUSE 挂载如何经由 CSI 注入 Pod）——最后这一问直接和 P1 现勘连起来。

降级路径：动态供给做不完就停在 static PV 闭环（八成弹药已到手）；在 kind 里注册不上，就在节点上用 `csc` 工具直接调你的 gRPC 方法来验证。

M4 vLLM token 级 profiling（v4.1 新增，P2 互补，1-2 天）

互补关系：P2 只有 vLLM /metrics 的**粗粒度**指标（TTFT/TBT/吞吐/前缀命中率）——M4 补的是 **token 级**观测，能看到每个 token 的 prefill/decode 时间、prefix hit/miss 精确到 token 位置。

做什么：用 eBPF uprobes + Python 对 vLLM 做 token 级 profile。- 用 `bpftrace` 或 `bcc` 的 uprobes 挂在 vLLM 的 Python 函数上（比如 `vllm/v1/core/async_scheduler.py` 里的 `schedule()` 方法、`gpu_model_runner.py` 里的 `execute_model()` 方法），或者用 `perf` 记录函数调用时间；- 对每个请求记录：每个 prefill chunk 的耗时、每个 decode step 的耗时、哪个 replica 做的、prefix hit depth 精确到 token 位置；- 用 Python 脚本解析采样数据，产出：**每个 token 的 prefill/decode 时间分布图、prefix hit/miss 的 token 位置热力图、preemption 发生在哪个 token 位置**。

> 不依赖 GPU：如果当前是 CPU 后端的 vLLM，直接跑 uprobes；如果是 GPU 后端，用 uprobes 挂在 host Python 函数上（不需要挂 GPU 内核）。

和 P2 的联动：P2 的 TTFT/TBT 是”整个请求的聚合指标”，M4 是”每个 token 的拆解指标”——当 P2 的 TTFT 异常高时，用 M4 可以定位是”某个 prefill chunk 特别慢”还是”prefix miss 导致 prefill 重新计算”。

M5 vLLM 引擎源码精读笔记（v4.1 新增，P2 互补，1-2 天）

互补关系：P2 消费的是 AIBrix 路由层源码（Envoy plugin / Go router），没有读 vLLM 引擎内核——M5 补的是 vLLM 引擎核心的 C++/Python 源码精读，和 S7 联动（S7 是”读完之后写简化版”，M5 是”精读之后写笔记”）。

做什么：精读 vLLM 源码的三个核心文件（用 vLLM v1.x 锁定版本），产出一篇半页笔记：1. **block_manager**：KV cache block 的分配/释放/淘汰/前缀索引——重点回答”vLLM 怎么判断这个 block 还能不能被 in-flight 请求引用”、“前缀命中深度是怎么从 block hash 链算出来的”2. **scheduler**：continuous batching + chunked prefill + preemption 的调度逻辑——重点回答”vLLM v1 怎么平衡 prefill 和 decode 的 GPU 时间”、“preemption 怎么选被驱逐的请求”3. **attention**：paged attention 的内存布局——重点回答”为什么 paged attention 要把 KV cache 分成固定大小的 block（比如 16 blocks/页）“、”token id 怎么映射到 block index”

和 P2 的联动：P2 的核心 2（弹性伸缩）需要理解”新副本需要预热多少 KV block 才能有意义地接受流量”——M5 的笔记里能找到 block 粒度和 prefix 命中深度的关系，直接给预热策略提供参数。

微弹药与面试域的关系（v4.1 扩展）

微弹药	覆盖的面试域	联动
M1	容器运行时、namespace/cgroup、CRI 链路	S6（K8s 的下层）、S2 的 overlayfs 坑
M2	CNI、容器网络模型、netns、iptables/IPVS	S6 调度之后”Pod 如何落地并联网”
M3	CSI、kubelet 卷生命周期、存储编排	P1 现勘（Fluid/JuiceFS CSI）、S4 存储岗叙事
M4	eBPF uprobes、token 级观测、profiling	P2 的 TTFT 归因、S7 的 token 级对比
M5	vLLM 引擎内核、block_manager、scheduler、attention	S7（简化版 KV manager 的设计依据）、P2（引擎内核视角）

---

v4.1 定稿，2026-10-15